

PROCESAMIENTO DE DATOS MASIVOS

Actividad 1 – Programación en Hadoop

Ejercicio 2

Enunciado

Dado el dataset que se adjunta a la actividad, que contiene los resultados de los partidos de la primera división de fútbol española, **crea un programa llamado `mediaGolesPartido` que para cada equipo indique la media de goles marcados por partido jugado en su estadio y como visitante, siguiendo el formato `equipo;local/visitante;mediagoles`**. La salida debe mostrar los equipos ordenados alfabéticamente. Se valorará positivamente la optimización del programa, por ejemplo minimizando el número de información que se transmite por el cluster. Debe implementar map, combine y reduce.

Solución

Se propone la solución contenida en la carpeta “ej2”, compuesta por los siguientes documentos:

- Datos de entrada: `resultados_futbol.csv`
- Código: se han desarrollado los scripts `mapper_mediaGolesPartido.py`, `combiner_mediaGolesPartido.py` y `reducer_mediaGolesPartido.py`.
- Comando de ejecución: el código a escribir en la terminal se reporta en `terminal_mediaGolesPartido.txt`
- Resultado: el resultado generado por el programa se reporta en la carpeta `salidaMediaGolesPartido`

Esta solución se ha ejecutado utilizando la versión 3.4.0 de Hadoop en un único nodo.

Código mapper

El mapper recibe todos los partidos jugados en la liga española de fútbol entre las temporadas 2000-2001 y 2023-2024 y emite como salida dos resultados por partido en formato ‘`equipo;rol\tgoles`’, donde `rol` puede ser Local o Visitante. De esta manera, esta manera, tendremos una clave en formato `equipo;rol` y un valor con los goles. Por ejemplo:

- Input: si recibe las líneas
 - 20,2000-2001,3,0-2,Valencia - Alavés,Valencia,Alavés
 - 216,2000-2001,22,0-1,Alavés - Valencia,Alavés,Valencia
 - 482,2001-2002,11,1-1,Alavés - Valencia,Alavés,Valencia
 - 671,2001-2002,30,1-2,Valencia - Alavés,Valencia,Alavés
- Output:
 - Valencia;Local 0
 - Alavés;Visitante 2
 - Alavés;Local 0
 - Valencia;Visitante 1
 - Alavés;Local 1
 - Valencia;Visitante 1
 - Valencia;Local 1

- Alavés;Visitante 2

Para cada línea o partido, el código retiene la información relevante para realizar los cálculos (resultado, que está en el tercer elemento; equipo local, elemento 5; y equipo visitante, elemento 6). Después, emite los resultados en formato 'equipo;rol\tgoles', para que puedan ser procesados por el combiner.

```
#!/usr/bin/python3
import sys

'''
Mapper de mediaGolesPartido

input: las líneas del csv
output: equipo;Local/Visitante\tgoles
'''

# por cada línea (partido):
# me quedo con la info. relevante (resultado, equipo local, eq. visitante)
# extraigo de resultado los goles marcados por cada equipo
# emito pares (equipo;rol(Local o Visitante)\tgoles), 2 resultados por partido

for linea in sys.stdin:
    linea = linea.strip()

    # Añadimos un bloque try-except para manejar líneas con formato incorrecto,
    # como la línea de cabecera o cualquier otra que no tenga los datos esperados.
    try: # me quedo con la información relevante
        linea_parse = linea.split(',')
        resultado = linea_parse[3]
        local = linea_parse[5]
        visitante = linea_parse[6]

        goles_local, goles_visit = resultado.split('-')
        goles_local = int(goles_local)
        goles_visit = int(goles_visit)

        # Normalizo Alavés y Real
        if local == "Deportivo Alavés":
            local = "Alavés"
        elif local == "R. Sociedad":
            local = "Real Sociedad"
        if visitante == "Deportivo Alavés":
            visitante = "Alavés"
        elif visitante == "R. Sociedad":
            visitante = "Real Sociedad"

        # emito 2 pares: clave (equipo; local/visitante), valor (goles)
        print(f"{local};Local\t{goles_local}")
        print(f"{visitante};Visitante\t{goles_visit}")

    except (IndexError, ValueError):
        continue
```

Código combiner

El combiner recibe, en cada nodo, los resultados emitidos por el mapper. Para cada equipo y rol (local o visitante), suma los goles marcados y los partidos jugados. Así, el combiner devuelve dos filas para cada equipo: una con los goles marcados y partidos jugados como local y otra con los goles marcados y partidos jugados como visitante. Siguiendo el ejemplo anterior:

- Input:
 - Valencia;Local 0
 - Alavés;Visitante 2

- Alavés;Local 0
- Valencia;Visitante 1
- Alavés;Local 1
- Valencia;Visitante 1
- Valencia;Local 1
- Alavés;Visitante 2
- Output:
 - Valencia;Local 1 2
 - Alavés;Visitante 4 2
 - Alavés;Local 1 2
 - Valencia;Visitante 2 2

Observamos que gracias al combiner, estaremos escribiendo en disco y enviando al reducer menos datos que si simplemente utilizáramos el mapper. En nuestro ejemplo, pasaríamos de escribir y enviar 8 datos a 4. En este caso, al no cumplir la media con las propiedades asociativa y distributiva, se ha definido un combiner diferente al reducer.

```
#!/usr/bin/python3
import sys

'''
Combiner de mediaGolesPartido

input: equipo;Local/Visitante\tgoles
output: equipo;Local/Visitante\tsuma_goles\tn_partidos
'''

subproblema = None
n_partidos = 0
suma_goles = 0

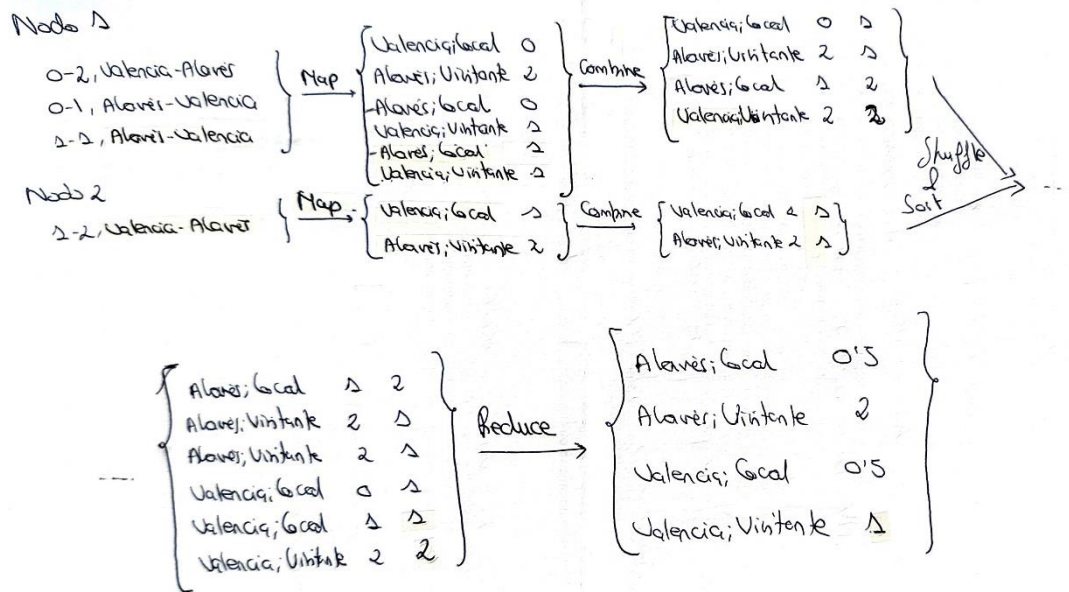
for claveValor in sys.stdin:
    equipo_rol, goles = claveValor.split('\t', 1)
    goles = int(goles) # por defecto era string
    if subproblema == None:
        subproblema = equipo_rol
    if subproblema == equipo_rol: # si sigo con ese equipo y rol, sumo
        n_partidos += 1
        suma_goles += goles
    else:
        print("%s\t%s\t%s" % (subproblema, suma_goles, n_partidos)) # emito total de equipo
        # y paso al siguiente equipo
        subproblema = equipo_rol
        n_partidos = 1
        suma_goles = goles
# emito el último equipo fuera del bucle
print("%s\t%s\t%s" % (subproblema, suma_goles, n_partidos)) # emito total de equipo
```

Código reducer

Al reducer le llegarán todos los resultados emitidos por todos los combiners en los diferentes nodos. Gracias al Shuffle and Sort, el reducer recibirá los datos de todos los nodos ordenados por clave alfabéticamente.

Se divide el problema original (calcular la media de goles marcados por cada equipo como local y como visitante) en subproblemas (para cada equipo y rol, calcular la media de goles marcados). El reducer emite dos resultados por equipo, uno con la media de goles marcados como local y otro con la media de goles marcados como visitante.

El siguiente diagrama muestra el proceso si los datos de nuestro ejemplo estuvieran distribuidos en dos nodos:



Ejecución

Seguimos los siguientes pasos para ejecutar el programa:

1. Ubicarnos en la ruta deseada (`cd /home/agorostizal/Desktop/practica1/ej2`)
2. Dar permiso de ejecución a los scripts (`chmod u+x mapper_mediaGolesPartido.py`, `chmod u+x combiner_mediaGolesPartido.py`, `chmod u+x reducer_mediaGolesPartido.py`)
3. Ejecutar el programa:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar
-files ./mapper_mediaGolesPartido.py,
./combiner_mediaGolesPartido.py,
./reducer_mediaGolesPartido.py
-mapper ./mapper_mediaGolesPartido.py
-reducer ./reducer_mediaGolesPartido.py
-combiner ./combiner_mediaGolesPartido.py
-input resultados_futbol.csv
-output ./salidaMediaGolesPartido
```

```
[agorostizal@vbox ej1]$ cd /home/agorostizal/Desktop/practical/ej2
[agorostizal@vbox ej2]$ chmod u+x mapper_mediaGolesPartido.py
[agorostizal@vbox ej2]$ chmod u+x combiner_mediaGolesPartido.py
[agorostizal@vbox ej2]$ chmod u+x reducer_mediaGolesPartido.py
[agorostizal@vbox ej2]$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar -
files ./mapper_mediaGolesPartido.py,./combiner_mediaGolesPartido.py,./reducer_mediaGolesPartido.py
-mapper ./mapper_mediaGolesPartido.py -reducer ./reducer_mediaGolesPartido.py -combiner ./combiner_
mediaGolesPartido.py -input resultados_futbol.csv -output ./salidaMediaGolesPartido
```

Y obtenemos la siguiente salida en terminal:

```

File System Counters
  FILE: Number of bytes read=1433670
  FILE: Number of bytes written=1767655
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
Map-Reduce Framework
  Map input records=9121
  Map output records=18240
  Map output bytes=360355
  Map output materialized bytes=2277
  Input split bytes=118
  Combine input records=18240
  Combine output records=84
  Reduce input groups=84
  Reduce shuffle bytes=2277
  Reduce input records=84
  Reduce output records=84
  Spilled Records=168
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=28
  Total committed heap usage (bytes)=358612992
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=569436
File Output Format Counters
  Bytes Written=2110
2025-06-28 17:38:30,717 INFO streaming.StreamJob: Output directory: ./salidaMediaGolesPartido

```

- El mapper ha recibido 9.121 filas y ha devuelto 18.240 (dos resultados por fila, excepto para la primera, ya que corresponde a los encabezados).
- El combiner ha recibido esos 18.240 datos, los ha combinado y ha devuelto 84 (dos por equipo con la suma de goles y partidos jugados como local y como visitante).
- El reducer ha recibido esos 82 datos y ha emitido 82 (la media de goles marcados como local y como visitante).

Los resultados se han escrito en la carpeta 'salidaMediaGolesPartido', disponible entre los materiales entregados en este ejercicio.

```

Alavés;Local;1.120
Alavés;Visitante;0.957
Albacete;Local;1.105
Albacete;Visitante;0.816
Almería;Local;1.263
Almería;Visitante;0.940
Athletic;Local;1.485
Athletic;Visitante;1.083
Atlético;Local;1.828
Atlético;Visitante;1.299
Barcelona;Local;2.671
Barcelona;Visitante;1.958
Celta;Local;1.442
Celta;Visitante;1.088
Cádiz;Local;1.013
Cádiz;Visitante;0.789
Córdoba;Local;0.632

```

Córdoba;Visitante;0.526
Deportivo;Local;1.511
Deportivo;Visitante;1.040
Eibar;Local;1.271
Eibar;Visitante;0.962
Elche;Local;0.968
Elche;Visitante;0.811
Espanyol;Local;1.371
Espanyol;Visitante;0.938
Getafe;Local;1.281
Getafe;Visitante;0.906
Gimnàstic Tarragona;Local;1.211
Gimnàstic Tarragona;Visitante;0.579
Girona;Local;1.368
Girona;Visitante;1.175
Granada;Local;1.111
Granada;Visitante;0.947
Huesca;Local;1.079
Huesca;Visitante;0.947
Hércules;Local;1.421
Hércules;Visitante;0.474
Las Palmas;Local;1.221
Las Palmas;Visitante;0.926
Leganés;Local;1.013
Leganés;Visitante;0.789
Levante;Local;1.312
Levante;Visitante;0.932
Mallorca;Local;1.452
Mallorca;Visitante;1.012
Málaga;Local;1.393
Málaga;Visitante;1.019
Numancia;Local;1.263
Numancia;Visitante;0.776
Osasuna;Local;1.235
Osasuna;Visitante;0.895
Racing;Local;1.241
Racing;Visitante;1.053
Rayo Vallecano;Local;1.404
Rayo Vallecano;Visitante;1.035
Real Betis;Local;1.358
Real Betis;Visitante;1.089
Real Madrid;Local;2.621
Real Madrid;Visitante;1.886
Real Murcia;Local;1.105
Real Murcia;Visitante;0.605
Real Oviedo;Local;1.711
Real Oviedo;Visitante;0.789
Real Sociedad;Local;1.551
Real Sociedad;Visitante;1.158
Real Sporting;Local;1.272
Real Sporting;Visitante;0.851
Real Valladolid;Local;1.162

Real Valladolid;Visitante;0.902
Real Zaragoza;Local;1.522
Real Zaragoza;Visitante;0.921
Recreativo;Local;1.211
Recreativo;Visitante;0.934
Sevilla;Local;1.723
Sevilla;Visitante;1.227
Tenerife;Local;1.237
Tenerife;Visitante;0.658
Valencia;Local;1.759
Valencia;Visitante;1.204
Villarreal;Local;1.670
Villarreal;Visitante;1.191
Xerez;Local;1.053
Xerez;Visitante;0.947